

TypeScript - Principiante

Tema 17: Módulos, import y export

20 ejercicios progresivos, sin soluciones, para practicar separación de código en archivos, exportaciones nombradas y por defecto, importaciones, alias de importación y organización de modelos, utilidades, repositorios y servicios.

Ejercicio 1 - Exportar una constante

Crea config.ts con una constante APP_NAME y exportala. Importala desde main.ts.

Requisitos

- Usa export nombrado.
- Muestra el valor desde main.ts.

Ejemplo de ejecucion

Aplicacion: Learn TypeScript

Objetivo

Practicar export e import basicos.

Pista

Exporta desde un archivo e importa usando llaves.

Ejercicio 2 - Exportar una funcion

Crea math.ts con una funcion add y utilizala desde main.ts.

Requisitos

- La funcion recibe dos number.
- Usa export nombrado.

Ejemplo de ejecucion

Resultado: 15

Objetivo

Separar funciones en modulos.

Pista

El archivo que consume la funcion debe importarla.

Ejercicio 3 - Varias funciones

Crea math.ts con add, subtract y multiply.

Requisitos

- Exporta las tres funciones.
- Importalas en una sola sentencia.
- Usa todas desde main.ts.

Ejemplo de ejecucion

Suma: 8 | Resta: 2 | Multiplicacion: 15

Objetivo

Practicar multiples exports nombrados.

Pista

Puedes importar varios nombres entre llaves.

Ejercicio 4 - Exportar un type

Crea models/user.ts con un type User y usalo desde main.ts.

Requisitos

- Exporta el type.
- Importalo usando import type cuando corresponda.
- Crea al menos dos usuarios.

Ejemplo de ejecucion

1 - Ana

Objetivo

Practicar modulos con tipos.

Pista

Los tipos tambien pueden exportarse e importarse.

Ejercicio 5 - Exportar una interface

Crea models/product.ts con una interface Product.

Requisitos

- Exporta Product.
- Usala para tipar un array en otro archivo.

Ejemplo de ejecucion

Productos: 3

Objetivo

Organizar modelos en archivos separados.

Pista

Una interface exportada puede utilizarse en cualquier modulo que la importe.

Ejercicio 6 - Exportar una clase

Crea classes/counter.ts con una clase Counter.

Requisitos

- Exporta la clase.
- Importala en main.ts.
- Crea dos instancias independientes.

Ejemplo de ejecucion

Contador A: 3 | Contador B: 1

Objetivo

Practicar clases entre modulos.

Pista

Cada archivo puede exponer las clases que otros necesitan.

Ejercicio 7 - Default export

Crea logger.ts con una funcion principal y exportala como default.

Requisitos

- Importala sin llaves.
- Usala desde main.ts.

Ejemplo de ejecucion

[INFO] Aplicacion iniciada

Objetivo

Practicar export default.

Pista

Una importacion default puede recibir el nombre local que elijas.

Ejercicio 8 - Named vs default

Crea un modulo que tenga un export default y dos exports nombrados.

Requisitos

- Importa los tres correctamente.
- Usa cada elemento.
- Escribe un comentario indicando la diferencia de sintaxis.

Ejemplo de ejecucion

Modulo cargado correctamente

Objetivo

Distinguir exportaciones default y nombradas.

Pista

La default se importa fuera de llaves; las nombradas dentro.

Ejercicio 9 - Alias de importacion

Exporta una funcion format desde un modulo e importala con otro nombre.

Requisitos

- Usa as en la importacion.
- Ejecuta la funcion usando el alias.

Ejemplo de ejecucion

Texto formateado

Objetivo

Practicar alias al importar.

Pista

Un alias evita conflictos de nombres locales.

Ejercicio 10 - Alias de exportacion

Crea una funcion interna calculateTotal y exportala con el nombre total.

Requisitos

- No cambies el nombre original de la funcion.
- Importa total desde otro archivo.

Ejemplo de ejecucion

Total: \$2500

Objetivo

Practicar renombrado al exportar.

Pista

Puedes usar export { nombre as otroNombre }.

Ejercicio 11 - Archivo index

Crea una carpeta models con User y Product y un index.ts que reexporte ambos.

Requisitos

- Importa los modelos desde models/index.ts o desde la ruta de la carpeta segun tu configuracion.
- No importes cada archivo directamente desde main.

Ejemplo de ejecucion

Modelos cargados

Objetivo

Introducir barrel files.

Pista

Un index puede centralizar reexportaciones.

Ejercicio 12 - Utilidades

Crea utils/string.ts y utils/number.ts con funciones diferentes.

Requisitos

- Crea utils/index.ts para reexportarlas.
- Usalas desde main.ts.

Ejemplo de ejecucion

OSCAR

Precio: \$850.00

Objetivo

Organizar utilidades por responsabilidad.

Pista

Evita colocar todas las funciones en un unico archivo grande.

Ejercicio 13 - Modelo y servicio

Crea User en models/user.ts y funciones de usuarios en services/user-service.ts.

Requisitos

- El servicio importa User.
- main.ts importa el servicio.
- Implementa una funcion para buscar por id.

Ejemplo de ejecucion

Usuario encontrado: Ana

Objetivo

Practicar dependencias entre modulos.

Pista

Los servicios pueden depender de los modelos.

Ejercicio 14 - Repositorio

Crea UserRepository en repositories/user-repository.ts.

Requisitos

- El repositorio usa User.
- Implementa getAll, getById y add.
- Usalo desde main.ts.

Ejemplo de ejecucion

Usuarios registrados: 4

Objetivo

Separar acceso a datos en un modulo.

Pista

Mantiene la coleccion interna dentro del repositorio.

Ejercicio 15 - Servicio y repositorio

Crea UserService que reciba UserRepository.

Requisitos

- Cada clase vive en su propio archivo.
- UserService importa el repositorio.
- main.ts construye las dependencias.

Ejemplo de ejecucion

Usuario creado correctamente

Objetivo

Practicar una estructura modular cercana a aplicaciones reales.

Pista

Cada modulo debe conocer solamente las dependencias que necesita.

Ejercicio 16 - Tipos compartidos

Crea types/common.ts con alias Id y Status.

Requisitos

- Usalos en varios modelos.
- Importa solo los tipos necesarios.
- No dupliques sus definiciones.

Ejemplo de ejecucion

ID: USR-10 | Estado: active

Objetivo

Centralizar tipos realmente compartidos.

Pista

No conviertas common.ts en un archivo para todo; coloca solo tipos reutilizados.

Ejercicio 17 - Configuración

Crea config/app-config.ts y exporta una configuración tipada.

Requisitos

- Define AppConfig.
- Exporta una instancia de configuración.
- Usala desde un servicio.

Ejemplo de ejecución

API: http://localhost:3000

Objetivo

Practicar módulos de configuración.

Pista

Separa la definición del tipo y el valor cuando sea útil.

Ejercicio 18 - Colisiones de nombres

Crea dos módulos que exporten una función llamada create.

Requisitos

- Importa ambas en el mismo archivo usando alias.
- Ejecuta las dos sin conflicto.

Ejemplo de ejecución

Usuario creado

Producto creado

Objetivo

Resolver conflictos de nombres entre módulos.

Pista

Renombra durante la importación con as.

Ejercicio 19 - Estructura por capas

Organiza un pequeño programa en models, repositories, services y utils.

Requisitos

- Cada responsabilidad debe estar en su archivo.
- Usa import type para imports que solo sean tipos cuando corresponda.
- Evita dependencias circulares.
- main.ts actua como punto de entrada.

Ejemplo de ejecucion

Aplicacion iniciada correctamente

Objetivo

Practicar organizacion modular.

Pista

Dibuja primero que modulo depende de cual.

Ejercicio 20 - Sistema de gastos modular

Construye un sistema de gastos dividido en modulos.

Requisitos

- Crea models/user.ts, models/category.ts y models/expense.ts.
- Crea repositories/expense-repository.ts.
- Crea services/expense-service.ts.
- Crea utils/currency.ts.
- Usa index.ts donde tenga sentido para reexportar.
- ExpenseService permite registrar, listar y totalizar gastos.
- El repositorio mantiene los datos.
- main.ts crea las dependencias y ejecuta ejemplos.
- Usa imports y exports correctamente.
- Evita dependencias circulares.
- No uses any.

Ejemplo de ejecucion

Gasto registrado

Gastos: 5

Total: \$3250.00

Objetivo

Integrar modulos, modelos, repositorios, servicios y utilidades.

Pista

Haz que cada archivo tenga una responsabilidad clara y exporte solamente lo necesario.